



Docker — Step 2

Building my own image — Dockerfile line-by-line, going inside containers, and the port puzzle

docker build

Dockerfile, line by line

-it / sh / exit

Layer caching

0.0.0.0 vs 127.0.0.1

Box colors: **Blue** = key idea · **Purple** = bonus detail · **Red** = a mistake I actually hit · **Green** = added after deeper discussion.

1 What Step 2 Is

In Step 1 I *borrowed* a finished image (nginx). In Step 2 I became the cook: my own code + a recipe (Dockerfile) → **docker build** → **my own image** → run a container from it.

```
STEP 1 (borrowing):  docker run nginx          → use someone else's image
STEP 2 (creating):   docker build → docker run  → use MY OWN image
```

File	Job
<code>index.js</code>	A tiny Express web server — my app (serves a 🐙 page on port 3000)
<code>package.json</code>	Lists my one dependency (Express)
<code>Dockerfile</code>	The recipe — how to bake my code into an image
<code>.dockerignore</code>	Keeps junk (like <code>node_modules</code>) out of the image

2 The Build Command & Tags

```
docker build -t myapp .
|      |      |
|      |      |
|      |      |
```

```
└─ "." = build using THIS folder (where the Dockerfile is)
└─ the NAME for the image
└─ -t = "tag" (name) the image
```

💡 A tag is really name:version

`-t myapp` silently becomes `myapp:latest`. The `:7` in `redis:7` and `:20-alpine` in `FROM node:20-alpine` are the same kind of tag — I'd been using tags all along.

3 The Dockerfile, Line by Line

✅ Added — the whole recipe explained

Docker reads the file top-to-bottom; each instruction is a **step** that adds a **layer**.

Line	What it does
<code>FROM node:20-alpine</code>	Start from a base image that already has Node.js v20 on a tiny Linux (alpine ≈ 5 MB). Always the first line.
<code>WORKDIR /app</code>	Work inside <code>/app</code> from now on (creates it). Like <code>cd /app</code> .
<code>COPY package*.json ./</code>	Copy <i>only</i> the dependency list first — for caching (see §8).
<code>RUN npm install</code>	Run a command during the build ; installs Express into the image.
<code>COPY . .</code>	Copy the rest of my code in (first <code>.</code> = my folder, second = <code>/app</code>).
<code>EXPOSE 3000</code>	Just documentation — a label saying "uses 3000." Does not open the port.
<code>CMD ["node","index.js"]</code>	The command that runs when a container starts . The container's main process.

🔑 RUN vs CMD — the distinction that confuses everyone

RUN = runs *now, while building the image* (e.g. install stuff) — happens once. **CMD** = runs *later, when a container starts* (e.g. launch the app) — every `docker run`. Think: RUN = "set up the kitchen", CMD = "cook when a customer arrives."

⚠️ EXPOSE ≠ opening a port

`EXPOSE 3000` is only a note for humans/tools. The **real** port opening is `-p 3000:3000` at `docker run`. I could delete the EXPOSE line and the app would behave identically. EXPOSE = the sign on the door; `-p` = actually unlocking it.

The whole file as a story

FROM = "give me a kitchen with Node" · WORKDIR = "I'll work at this counter" · COPY package = "here's my shopping list" · RUN install = "stock the ingredients now" · COPY . . = "here's my actual food" · EXPOSE = "put up a sign: window 3000" · CMD = "when a customer comes, cook THIS."

4 Why WORKDIR Matters

✓ Added — what happens without it

WORKDIR isn't strictly required — the app would still run. But without it the default folder is the container's **system root** `/`, which is NOT empty.

```
WITHOUT WORKDIR:
/
├─ bin/  etc/  lib/  usr/  var/
├─ index.js      🤔 mixed in
├─ package.json  🤔
└─ node_modules/ 🤔

WITH WORKDIR /app:
/
├─ bin/  etc/  lib/  usr/  var/  (system, untouched)
└─ app/      📦 all MINE, neatly
    ├─ index.js
    ├─ package.json
    └─ node_modules/
```

Without WORKDIR, the risk...

Why it's bad

Mess / no isolation

My files mingle with OS files; hard to find anything

Name collisions

A folder named like a system one could clash with `/` overwrite system files

Fragile paths & debugging

Harder to reason about; have to hunt through `/` to find my 3 files

One-liner: without WORKDIR, my app is dumped into the system root and mixes with the OS's own files — usually still runs, but messy, fragile, and risky. **WORKDIR /app** gives it a clean room.

5 Building & Running My Image

```
[1/5] FROM node:20-alpine
[2/5] WORKDIR /app
[3/5] COPY package*.json ./
[4/5] RUN npm install
[5/5] COPY . .
...exporting to image → naming to myapp:latest ✓
```

```
docker run -d -p 3000:3000 --name myapp-container myapp # run a container from my
image
curl http://localhost:3000 # → my 🐙 page
docker images # → "myapp" sits alongside nginx & node
```

6 Going Inside a Container — `-it`, `sh`, `exit`

✓ Added — what `-it` actually is

It's **two flags**: `-i` = **interactive** (let me type into it) and `-t` = **TTY** (give me a proper terminal with a prompt). They travel together whenever I want to go inside a container. `sh` = a **shell** (command line) inside the container — I use `sh` not `bash` because alpine is too minimal to include bash.

```
docker run -it myapp sh # start a NEW container, drop into a shell inside
docker exec -it myapp sh # step into an ALREADY-RUNNING container
```

Command	Use
<code>docker run -it img sh</code>	Create a brand-new container and go inside
<code>docker exec -it name sh</code>	Jump into an existing, running container
<code>exit</code> (or Ctrl+D)	Leave the shell → back to my Mac

🚫 Gotcha — READ YOUR PROMPT (I got badly confused here)

I kept pasting `docker` commands while still *inside* a container and got errors like `sh: docker: not found`. **docker does not exist inside a container** — it only runs on my Mac.

Prompt looks like...	I am...	docker works?
<code>/app #</code>	INSIDE a container	✗ No

💡 The container dies when I exit

If `sh` is the container's main process, typing `exit` stops the container (a container lives only as long as its main process). Then `docker exec` into it fails with *"container is not running"* — I'd have to `docker start` it first, or just `docker run` a fresh one. Also: **Ctrl+C** stops the running program; `exit` leaves the shell.

7 Anything After the Image Name Overrides CMD

✔ Added — a hidden but powerful behavior

Whatever I type after the image name in `docker run` **replaces the Dockerfile's CMD**. So `docker run myapp sh` runs a shell *instead of* my app — meaning **the web server does NOT start**. Great for poking around an image without launching it.

Command	Runs inside	App running?
<code>docker run -p 3000:3000 myapp</code>	<code>node index.js</code> (the CMD)	✔ Yes
<code>docker run -it myapp sh</code>	<code>sh</code> (CMD overridden)	✗ No — just a shell
<code>docker run myapp ls</code>	<code>ls</code> , then exits	✗ No

8 The Daily Loop & Layer Caching

edit code → docker build → docker rm -f → docker run → test

⚠ A rebuild does NOT update a running container

After editing code and rebuilding, the browser still showed the OLD message. A new image never auto-updates a running container — I must recreate it: `docker rm -f myapp-container` then `docker run ... myapp` again.

Why my rebuild took ~2 seconds: CACHED

```
CACHED [2/5] WORKDIR /app
CACHED [3/5] COPY package*.json ./
CACHED [4/5] RUN npm install      # ← the slow step, SKIPPED!
[5/5] COPY . .                  # ← re-ran (my code changed)
```

Each line is a layer. On rebuild Docker reuses every layer that's unchanged. **The cache breaks at the first changed step and stays broken for every step after it.** I changed only `index.js`, which the final `COPY . .` picks up — so only that cheap layer rebuilt.

✅ Why the COPY order is deliberate

Code changes constantly; dependencies rarely. Copying `package.json` + running `npm install` before `COPY . .` keeps the slow install **above** my changing code — so editing `index.js` doesn't re-run install. Reverse the order and every tiny edit would re-install everything.

9 The Port Puzzle — Listening vs Connecting

✅ Added — two different "port 3000"s

The `3000` comes from my **code** (`index.js`), not from `docker run . app.listen(3000)` opens a **real** listening socket *inside* the container (the `console.log` just reports it). Whether I can *reach* it from my Mac is a **separate** thing, controlled by `-p`.

Layer	What	From
Listening on a port	App opens 3000 <i>inside</i> the container	<code>app.listen</code> in <code>index.js</code>
Publishing a port	A doorway from my Mac → the container	<code>-p</code> at <code>docker run</code>

0.0.0.0 vs 127.0.0.1 — listen vs connect

🔑 They play opposite roles

`0.0.0.0` is a "listen on ALL my IPv4 addresses" wildcard — used when a server *binds*. You never *connect* to `0.0.0.0`. `127.0.0.1` is a **real, specific address** (loopback) — used when a client *connects*. And `0.0.0.0` includes `127.0.0.1`, so connecting to `127.0.0.1` reaches a server bound to `0.0.0.0`. ✅

🚫 Gotcha — wget localhost refused even though the server was up

From inside the container, `wget localhost:3000` → *Connection refused*, but `ps` showed `node index.js` running! Cause: my app listens on `0.0.0.0` = **IPv4 only**, but `localhost` can resolve to `::1` (IPv6) first → wrong door → refused. Using `127.0.0.1` (IPv4) works. (*It worked from my Mac browser earlier because Docker's `-p` forwarding handled IPv4 correctly.*)

Connect to...	Resolves to	Server (0.0.0.0=IPv4) listening?	Result
<code>127.0.0.1</code>	IPv4 loopback	✅ yes	works
<code>localhost</code>	maybe <code>::1</code> (IPv6)	❌ no	refused

Why bind to `0.0.0.0` at all? So the app answers on *all* doors — both internal loopback and the container's network IP that Docker's `-p` forwards to. Binding to only `127.0.0.1` would make `-p` unable to reach it. It's the standard for containerized apps.

10 Managing Images — rmi, Untagged vs Deleted

✅ Added — what `docker rmi myapp` showed

```
Untagged: myapp:latest      # removed the NAME pointing at the image
Deleted: sha256:6cb770a28... # deleted the actual image DATA
```

Running `docker rmi myapp:latest` again then said "*No such image*" — not a failure; it was **already gone** (the first command deleted it, since `myapp` = `myapp:latest`).

- **Untagged** = removed a name pointing at the image. **Deleted** = removed the actual data. Docker only deletes data once *no* names point to it.
- `docker rm` removes a **container**; `docker rmi` removes an **image**.
- The "U" / In Use column in `docker images` means a container still exists from that image — Docker won't delete it until those containers are removed (or use `-f`).

11 Every Command I Used in Step 2

```

docker build -t myapp . # bake my own image
docker run -d -p 3000:3000 --name myapp-container myapp # run it (door open)
docker run -it myapp sh # NEW container, shell inside
(CMD overridden)
docker exec -it test sh # step into a RUNNING container
ps # (inside) list processes - saw
"node index.js"
wget -q0- http://127.0.0.1:3000 # (inside) IPv4 → works
exit # leave the shell
docker images # list images (+ "U" In-Use
column)
docker ps / docker ps -a # running / all containers
docker logs <name> # view a container's output
docker rm -f <name> # remove a container
docker rmi myapp # remove an image (Untagged +
Deleted)
curl http://localhost:3000 # (Mac) honest test

```

✅ One-line summary of Step 2

I built my **own** image, read the **Dockerfile line by line** (incl. RUN-vs-CMD and EXPOSE-vs- **-p**), learned to go **inside** with **-it sh** (and that the prompt tells me where I am), saw that args **override CMD**, lived the **edit→build→recreate** loop with **layer caching**, and untangled the **listen (0.0.0.0) vs connect (127.0.0.1)** port puzzle. **Next: Step 3 — Docker Compose (my app + Redis).**